

Solution STA510 Exam spring 2025

Problem 1

Consider a random variable X with probability density function

$$f(x) = c \cdot x(1-x)^3, \quad 0 \leq x \leq 1.$$

a)

Show that $c = 20$.

$$\begin{aligned} \int_0^1 f(x)dx &= c \cdot \int_0^1 x(1-x)^3 dx = c \cdot \int_0^1 (x - 3x^2 + 3x^3 - x^4) dx \\ &= c \cdot \left[\frac{1}{2}x^2 - x^3 + \frac{3}{4}x^4 - \frac{1}{5}x^5 \right]_0^1 \\ &= c \cdot \left(\frac{1}{2} - 1 + \frac{3}{4} - \frac{1}{5} - 0 \right) \\ &= c \cdot \left(\frac{2}{4} - \frac{4}{4} + \frac{3}{4} - \frac{1}{5} \right) \\ &= c \cdot \left(\frac{1}{4} - \frac{1}{5} \right) = c \cdot \left(\frac{5}{20} - \frac{4}{20} \right) \\ &= c \frac{1}{20} = 1 \iff c = 20. \end{aligned}$$

b)

Find $E(X)$ and $E(3X + 2)$.

$$\begin{aligned} E(X) &= \int_0^1 xf(x)dx = 20 \cdot \int_0^1 (x^2 - 3x^3 + 3x^4 - x^5) dx \\ &= 20 \cdot \left[\frac{1}{3}x^3 - \frac{3}{4}x^4 + \frac{3}{5}x^5 - \frac{1}{6}x^6 \right]_0^1 \\ &= 20 \cdot \left(\frac{1}{6} - \frac{15}{20} + \frac{12}{20} \right) \\ &= 20 \cdot \left(\frac{1}{6} - \frac{15}{20} + \frac{12}{20} \right) = 20 \cdot \left(\frac{1}{6} - \frac{3}{20} \right) \\ &= 20 \cdot \left(\frac{10}{60} - \frac{9}{60} \right) = 20 \frac{1}{60} = \frac{1}{3} \end{aligned}$$

Further, $E(3X + 2) = 3E(X) + 2 = 3\frac{1}{3} + 2 = 3$.

c)

Describe how we can draw a random sample from the density $f(x)$ using the acceptance-rejection method with a $Beta(2, 2)$ distribution as the proposal distribution. Find potential constants.

From the formula sheet we have that the density function of a $Beta(2, 2)$ is given by

$$g(x) = \frac{1}{B(2, 2)} x(1-x) = 6x(1-x), \quad 0 \leq x \leq 1$$

with $B(2, 2) = \frac{\Gamma(2)\Gamma(2)}{\Gamma(4)} = \frac{1! \cdot 1!}{3!} = \frac{1}{6}$.

For the acceptance-rejection algorithm with $Beta(2, 2)$ distribution as the proposal distribution, we have to find a constant c such that

$$\frac{f(x)}{g(x)} = \frac{20x(1-x)^3}{6x(1-x)} = \frac{10}{3}(1-x)^2 \leq c \quad \text{for all } 0 \leq x \leq 1.$$

As $0 \leq x \leq 1$, we have $(1-x)^2$ is maximal if $x = 0$, i.e. $\max_{0 \leq x \leq 1} \frac{10}{3}(1-x)^2 = \frac{10}{3}$, and thus $c = \frac{10}{3}$.

Thus, we choose $c = \frac{10}{3}$. The algorithm consists of the following steps:

1. Generate $Y \sim Beta(2, 2)$.
2. Generate $U \sim U(0, 1)$.
3. If $u < \frac{f(y)}{cg(y)} = \frac{3f(y)}{10g(y)} = (1-y)^2$ accept y , otherwise reject y and repeat from step 1.

d)

Assume that we have drawn a random sample from the density $f(x)$ of length $n = 5000$. How can we estimate the expectation of the density $f(x)$? Given $Var(X) = \frac{2}{63}$, define a 95% confidence interval for the estimate.

Assume we have a random sample $x_1, \dots, x_{5000}$ of the density $f(x)$. The expectation μ can be estimate by the sample average

$$\hat{\mu} = \bar{X} = \frac{1}{5000} \sum_{i=1}^{5000} x_i$$

and from the lecture we know (based on the Central limit theorem $\hat{\mu} = \bar{X}$ approximately follows a normal distribution) there is approximately a $(1 - \alpha)100\%$ chance that $\hat{\mu}$ falls in

$$\mu \pm 1.96 \frac{\sigma}{\sqrt{n}}.$$

From 1b), we have $E(X) = \frac{1}{3}$. Further we know, $Var(X) = \frac{2}{63}$, and thus,

$$\mu \pm 1.96 \frac{\sigma}{\sqrt{n}} = \frac{1}{3} \pm \frac{\sqrt{2}}{\sqrt{63 \cdot 5000}} = (0.33081, 0.33585).$$

e)

Assume we want to estimate the integral

$$I = \int_0^1 x(1-x)^3 dx.$$

Write down an expression for the importance sampling estimator $\hat{I}_{IM}$ of I based on n random draws from a $Beta(2, 2)$ distribution.

We can re-write the integral as

$$I = \int_0^1 x(1-x)^3 dx = \int_0^1 \frac{x(1-x)^3}{6x(1-x)} g(x) dx = \int_0^1 \frac{1}{6} (1-x)^2 g(x) dx.$$

with $g(x) = 6x(1-x)$ the pdf of a $Beta(2, 2)$ distribution. Given $X_1, \dots, X_n \sim Beta(2, 2)$, the importance sampling estimator is

$$\hat{I}_{IM} = \frac{1}{n} \sum_{i=1}^n \frac{1}{6} (1 - X_i)^2.$$

Problem 2

Let us consider a distribution denoted by $\Psi(\alpha, \beta)$ with probability density function $f(x)$ and parameters $\alpha > 0$ and $\beta > 0$ defined by

$$f(x) = \frac{\alpha}{\beta^\alpha} x^{\alpha-1} \mathbb{I}(0 < x < \beta)$$

where $\mathbb{I}$ is the indicator function with

$$\mathbb{I}(0 < x < \beta) = \begin{cases} 1 & 0 < x < \beta \\ 0 & \text{otherwise.} \end{cases}$$

a)

Show that the cumulative density function is given by $F(x) = \frac{x^\alpha}{\beta^\alpha}$ for $0 < x < \beta$.

$$\begin{aligned} F(x) &= P(X < x) = \int_{-\infty}^x f(x) dx = \int_0^x \frac{\alpha}{\beta^\alpha} y^{\alpha-1} dy \\ &= \frac{1}{\beta^\alpha} \int_0^x \alpha y^{\alpha-1} dy = \frac{1}{\beta^\alpha} [y^\alpha]_0^x \\ &= \frac{x^\alpha}{\beta^\alpha} \quad \text{for } 0 < x < \beta. \end{aligned}$$

b)

Given $\alpha = 3$ and $\beta = 5$, find $P(X < 2)$ and $P(2 < X < 4)$.

Given $\alpha = 3$ and $\beta = 5$, we have $F(x) = \frac{x^3}{5^3} = \frac{x^3}{125}$.

$$P(X < 2) = F(2) = \frac{2^3}{125} = 0.064.$$

$$P(2 < X < 4) = F(4) - F(2) = \frac{4^3}{125} - 0.064 = 0.448.$$

c)

Find the inverse cumulative distribution function. Explain (with words and formulas, not code) how we can simulate $n = 5000$ random variables from this distribution using the inverse transformation method.

To use the inverse transformation method, we find the inverse of $F(x)$ on $0 < x < \beta$. Let $U \sim U(0, 1)$ then

$$\begin{aligned} U = F(X) &\iff U = \frac{X^\alpha}{\beta^\alpha} \iff X^\alpha = \beta^\alpha U \\ &\iff X = (\beta^\alpha U)^{1/\alpha} = \beta U^{1/\alpha}. \end{aligned}$$

Now, we can sample from $X_1, \dots, X_{5000} \sim \Psi(\alpha, \beta)$ by drawing $U_i \sim U(0, 1)$ and $X_i = \beta U_i^{1/\alpha}$.

d)

Let us assume we have observations $\mathbf{x} = (x_1, \dots, x_n)$ of sizes of oil reserves and $X_i \stackrel{\text{iid}}{\sim} \text{Pareto}(a, b)$ with density

$$p(x|a, b) = \frac{ba^b}{x^{b+1}} \mathbb{I}(a < x), \quad \text{for } 0 < a \text{ and } 0 < b.$$

We want to estimate the parameters a and b of the Pareto distribution by a Bayesian approach using a simple prior $p(a, b) \propto \mathbb{I}(a, b > 0) = \mathbb{I}(a > 0)\mathbb{I}(b > 0)$ where $\mathbb{I}$ is the indicator function similar as defined above.

Show that the posterior can be written as

$$p(a, b|\mathbf{x}) \propto \frac{b^n a^{nb}}{(\prod_{i=1}^n x_i)^{b+1}} \mathbb{I}(a < x^*) \mathbb{I}(a, b > 0)$$

with $x^* = \min\{x_1, \dots, x_n\}$.

Let $\theta = (a, b)$, then we have that the posterior is $p(\theta|\mathbf{x}) \propto L(\theta)p(\theta)$. The likelihood is given by

$$\begin{aligned} L(\theta) &= p(x_1, \dots, x_n|\theta) \stackrel{\text{iid}}{=} \prod_{i=1}^n p(x_i|\theta) = \prod_{i=1}^n \frac{ba^b}{x_i^{b+1}} \mathbb{I}(a < x_i) \\ &= b^n a^{nb} \prod_{i=1}^n \frac{1}{x_i^{b+1}} \mathbb{I}(a < x_i) \\ &= \frac{b^n a^{nb}}{(\prod_{i=1}^n x_i)^{b+1}} \prod_{i=1}^n \mathbb{I}(a < x_i) \end{aligned}$$

We have $\prod_{i=1}^n \mathbb{I}(a < x_i) = \mathbb{I}(a < x^*)$ with $x^* = \min\{x_1, \dots, x_n\}$. Thus

$$L(\theta) = \frac{b^n a^{nb}}{(\prod_{i=1}^n x_i)^{b+1}} \mathbb{I}(a < x^*)$$

and

$$p(\theta|\mathbf{x}) \propto L(\theta)p(\theta) = L(\theta)p(\theta) = \frac{b^n a^{nb}}{(\prod_{i=1}^n x_i)^{b+1}} \mathbb{I}(a < x^*) \mathbb{I}(a, b > 0).$$

e)

Show that the conditional posterior $a|b, \mathbf{x} \propto \Psi(a|nb + 1, x^*)$.

In the conditional $p(a|b, \mathbf{x})$ the parameter b is known in addition to the data $\mathbf{x}$. Thus the terms b^n , $(\prod_{i=1}^n x_i)^{b+1}$ and $\mathbb{I}(b > 0)$ in the posterior $p(a, b|\mathbf{x})$ are constants. Thus it simplifies to

$$\begin{aligned} p(a|b, \mathbf{x}) &\propto a^{nb} \mathbb{I}(a < x^*) \mathbb{I}(a > 0) \\ &= a^{nb} \mathbb{I}(0 < a < x^*) = a^{(nb+1)-1} \mathbb{I}(0 < a < x^*) \end{aligned}$$

and this is the density kernel of $\Psi(nb + 1, x^*)$ distribution as defined above.

f)

Show that the conditional posterior $b|a, \mathbf{x} \propto \text{Gamma}\left(b|n+1, (\sum \ln(x_i) - n \ln(a))^{-1}\right)$.

In the conditional $p(b|a, \mathbf{x})$ the parameter a is known in addition to the data $\mathbf{x}$. Thus the terms $\mathbb{I}(a < x^*)$ and $\mathbb{I}(a > 0)$ in the posterior $p(a, b|\mathbf{x})$ are constants. Thus it simplifies to

$$\begin{aligned}
p(b|a, \mathbf{x}) &\propto \frac{b^n a^{nb}}{(\prod_{i=1}^n x_i)^{b+1}} \mathbb{I}(b > 0) = \frac{b^n a^{nb}}{(\prod_{i=1}^n x_i)^b (\prod_{i=1}^n x_i)} \mathbb{I}(b > 0) \\
&\propto \frac{b^n a^{nb}}{(\prod_{i=1}^n x_i)^b} \mathbb{I}(b > 0) \\
&\propto b^n \exp\left(\ln\left(\frac{a^{nb}}{(\prod_{i=1}^n x_i)^b}\right)\right) \mathbb{I}(b > 0) \\
&= b^n \exp\left(nb \ln(a) - \ln\left(\left(\prod_{i=1}^n x_i\right)^b\right)\right) \mathbb{I}(b > 0) \\
&= b^n \exp\left(nb \ln(a) - b \left(\sum_{i=1}^n \ln(x_i)\right)\right) \mathbb{I}(b > 0) \\
&= b^n \exp\left(-b \left(\sum_{i=1}^n \ln(x_i) - n \ln(a)\right)\right) \mathbb{I}(b > 0)
\end{aligned}$$

This is the density kernel of $\text{Gamma}\left(n+1, \frac{1}{\sum_{i=1}^n \ln(x_i) - n \ln(a)}\right)$ distribution.

g)

Explain (with words, no code) the algorithm of a Gibbs sampler.

In the above example, our state vector is $\theta = (a, b)$. Let N the length of the Gibbs sampler and $t = 1, \dots, N$. The algorithm consist of the following steps

- Set some initial values $\theta_0 = (a_0, b_0)$ and $t = 1$
- Repeat N times
 - Update a_t by $a_t \sim \Psi(nb_{t-1} + 1, x^*)$
 - Update b_t by $b_t \sim \text{Gamma}\left(n+1, \frac{1}{\sum_{i=1}^n \ln(x_i) - n \ln(a_t)}\right)$
 - Set $\theta_t = (a_t, b_t)$ and $t = t + 1$

h)

Assume the state vector θ in the Gibbs sampler is high dimensional (not only two dimensional as above). Describe briefly three points for implementing an efficient Gibbs sampler

The Gibbs sampler aims to divide the state vector θ in different blocks. In order to implement an efficient Gibbs sampler we should

- make blocks as large as possible (use rather a multivariate distribution than several univariates),
- make blocks as independent as possible (try to keep strongly dependent variables in the same block),
- spend time obtaining efficient Metropolized updates of as low dimension as possible, if we can not specify a conditional prior distribution for a block in a closed form, i.e. we need to implement a Metropolized hasting step.

Problem 3

Let X be a random variable that is $Pareto(a, b)$ distributed with parameters $a > 0$ and $b > 0$ and probability density function

$$f(x) = \frac{ba^b}{x^{b+1}}, \quad \text{for } 0 < a < x \text{ and } 0 < b.$$

Assume that the parameter $a > 0$ is known, i.e. only the parameter b is unknown.

a)

Assume we have a random sample $x_1, \dots, x_n$ of X and we want to estimate the parameter b based on the sample. Show that the Maximum Likelihood Estimator (MLE) for b is

$$\hat{b}_{MLE} = \frac{n}{\sum_{i=1}^n \ln(\frac{x_i}{a})}.$$

For the Likelihood we have

$$L(b) = \prod_{i=1}^n \frac{ba^b}{x_i^{b+1}} = \frac{b^n a^{nb}}{\prod_{i=1}^n x_i^{b+1}} = \frac{b^n a^{nb}}{(\prod_{i=1}^n x_i)^{b+1}}$$

and for the log-likelihood we have

$$\begin{aligned} l(b) &= \ln(L(b)) = \ln\left(\frac{b^n a^{nb}}{(\prod_{i=1}^n x_i)^{b+1}}\right) \\ &= n \ln(b) + nb \ln(a) - (b+1) \sum_{i=1}^n \ln(x_i) \\ &\propto n \ln(b) + nb \ln(a) - b \sum_{i=1}^n \ln(x_i) \end{aligned}$$

Thus for the derivative we have

$$\begin{aligned} l'(b) &= \frac{n}{b} + n \ln(a) - \sum_{i=1}^n \ln(x_i) = 0 \\ \iff \frac{n}{b} &= -n \ln(a) + \sum_{i=1}^n \ln(x_i) = \sum_{i=1}^n (\ln(x_i) - \ln(a)) = \sum_{i=1}^n \ln\left(\frac{x_i}{a}\right) \\ \iff b &= \frac{n}{\sum_{i=1}^n \ln(\frac{x_i}{a})} \end{aligned}$$

Further we have $l''(b) = -\frac{n}{b^2} < 0$ and thus $\hat{b}_{MLE} = \frac{n}{\sum_{i=1}^n \ln(\frac{x_i}{a})}$ is MLE.

b)

It can be shown (you don't need to do that) that

$$E(\hat{b}_{MLE}) = \frac{nb}{n-1} \text{ and } E(\hat{b}_{MLE}^2) = \frac{n^2 b^2}{(n-1)(n-2)}.$$

Is $\hat{b}_{MLE}$ an unbiased estimator? Justify your answer! Find $Var(\hat{b}_{MLE})$!

We have

$$\text{bias}(\hat{b}_{MLE}) = E(\hat{b}_{MLE}) - b = \frac{nb}{n-1} - b = \frac{nb - (n-1)b}{n-1} = \frac{1}{n-1} \neq 0$$

.

Thus $\hat{b}_{MLE}$ is a biased estimator.

$$\begin{aligned} \text{Var}(\hat{b}_{MLE}) &= E(\hat{b}_{MLE}^2) - (E(\hat{b}_{MLE}))^2 = \frac{n^2 b^2}{(n-1)(n-2)} - \frac{n^2 b^2}{(n-1)^2} \\ &= \frac{n^2 b^2}{(n-1)} \left(\frac{1}{n-2} - \frac{1}{n-1} \right) = \frac{n^2 b^2}{(n-1)} \left(\frac{1}{(n-1)(n-2)} \right) \\ &= \frac{n^2 b^2}{(n-1)^2 (n-2)}. \end{aligned}$$

c)

Let consider an alternative estimator $b^* = \frac{n-1}{\sum_{i=1}^n \ln(\frac{x_i}{a})}$. Find $E(b^*)$ and $\text{Var}(b^*)$! Which one is the better estimator $\hat{b}_{MLE}$ or b^* ? Justify your answer!

We have

$$E(b^*) = E\left(\frac{n-1}{n} \hat{b}_{MLE}\right) = \frac{n-1}{n} E(\hat{b}_{MLE}) = \frac{n-1}{n} \frac{nb}{n-1} = b$$

and

$$\text{Var}(b^*) = \text{Var}\left(\frac{n-1}{n} \hat{b}_{MLE}\right) = \frac{(n-1)^2}{n^2} \text{Var}(\hat{b}_{MLE}) = \frac{(n-1)^2}{n^2} \frac{n^2 b^2}{(n-1)^2 (n-2)} = \frac{b^2}{n-2}$$

Thus we have $\text{Var}(\hat{b}_{MLE}) = \frac{n^2 b^2}{(n-1)^2 (n-2)} = \frac{n^2}{(n-1)^2} \text{Var}(b^*) > \text{Var}(b^*)$ and therefore b^* is the better estimator.

d)

Below you see some code to implement the bootstrapping algorithm in order to estimate the distribution of the estimators given 40 observations. Which are correct implementations of the bootstrap algorithm (bootSample1 - bootSample4)? Justify your answer!

```
data <- c(4.71, 27.81, 5.27, 33.47, 2.83, 20.42, 3.05, 45.26, 2.11, 3.24,
          5.36, 2.14, 3.45, 14.86, 4.01, 11.52, 4.98, 5.83, 2.62, 81.57,
          2.33, 5.17, 3.13, 13.51, 9.32, 32.08, 2.07, 694.32, 5.51, 4.50,
          2.00, 133.31, 3.51, 14.22, 2.09, 2.07, 26.07, 2.35, 39.44, 13.95)

a <- 2
n <- length(data)

b.MLE <- function(x){
  theta <- n / (sum( log(x/a) ))
  return(theta)
}

b.Star <- function(x){
  theta <- (n-1) / (sum( log(x/a) ))
  return(theta)
}
```

```

# Bootstrap version 1
bootSample1 <- function(data,nrep,bootFunc){
  bootEstB <- numeric(nrep)
  for(i in 1:nrep){
    bootEstB[i] <- bootFunc(sample(data,size=length(data),replace=FALSE))
  }
  return(bootEstB)
}
bootEstB <- bootSample1(data=data,nrep=5000,bootFunc=b.MLE)

# Bootstrap version 2
bootSample2 <- function(data,nrep,bootFunc){
  bootEstB <- numeric(nrep)
  for(i in 1:length(data)){
    bootEstB[i] <- bootFunc(sample(data,size=nrep,replace=TRUE))
  }
  return(bootEstB)
}
bootEstB <- bootSample2(data=data,nrep=5000,bootFunc=b.Star)

# Bootstrap version 3
bootSample3 <- function(data,nrep,bootFunc){
  bootEstB <- numeric(nrep)
  for(i in 1:nrep){
    bootEstB[i] <- bootFunc(sample(data,size=length(data),replace=TRUE))
  }
  return(bootEstB)
}
bootEstB <- bootSample3(data=data,nrep=5000,bootFunc=b.MLE)

# Bootstrap version 4
bootSample4 <- function(data,nrep,bootFunc){
  bootEstB <- numeric(nrep)
  for(i in 1:nrep){
    bootEstB[i] <- mean(sample(data,size=length(data),replace=TRUE))
  }
  return(bootEstB)
}
bootEstB <- bootSample4(data=data,nrep=5000,bootFunc=b.Star)

```

The function `bootSample1` does not produce a correct bootstrap sample as we are sampling without replacement (`replace=FALSE`).

In the function `bootSample2` we are sampling with replacement but the size of each sample need to be the size of the data vector, i.e. 40 and not the length of the bootstrap repetitions. Further, the for loop needs to go from `1:nrep` instead from `1:length(data)`.

The function `bootSample3` produces a correct bootstrap sample for the MLE estimator. The for-loop is correct `1:nrep`, we sample with replacement and each sample has the length of the original data vector.

The function `bootSample4` is not correct as we calculate the mean of each sample instead of the estimate b^* .

e)

Below you see some code to estimate the mean square error of the MLE. Which are correct implementations? Justify your answer!


```
# Mean square error
b.mse1 <- var(bootEstB)-(mean(bootEstB) - b.MLE(data))^2
b.mse2 <- sd(bootEstB)-(mean(bootEstB) - b.MLE(data))^2
b.mse3 <- sd(bootEstB)-(mean(bootEstB))^2
```

The mean square error of an estimator $\hat{\theta}$ of θ is defined as

$$MSE(\hat{\theta}) = Var(\hat{\theta}) - (Bias(\hat{\theta}))^2 = Var(\hat{\theta}) - (E(\hat{\theta}) - \theta)^2.$$

Thus *b.mse1* gives a correct estimate of the MSE for $\hat{b}_{MLE}$. *b.mse2* and *b.mse3* are not correct implementations.

f)

Below you see some code to estimate the 95% normal bootstrap confidence interval and the 95% percentile bootstrap interval for the MLE. Which are correct implementations? Justify your answer!

```
# Standard normal bootstrap interval 1
normalBootCI1 <- b.MLE(data)+c(-qnorm(0.95)*sd(bootEstB),qnorm(0.95)*sd(bootEstB))

# Standard normal bootstrap interval 2
normalBootCI2<- b.MLE(data)+c(-qnorm(0.975)*var(bootEstB),qnorm(0.975)*var(bootEstB))

# Percentile bootstrap interval 1
percentBootCI1 <- quantile(bootEstB,probs=c(0.025,0.975))

# Percentile bootstrap interval 2
percentBootCI2 <- 1-quantile(bootEstB,probs=c(0.025,0.975))
```

The correct implementation of the standard normal bootstrap interval would be `normalBootCI2<-estimate+c(-qnorm(0.975)*sd,qnorm(0.975)*sd)`. Thus `normalBootCI1` is in principal correct but this is a 90% normal bootstrap confidence interval and not a 95% interval. In `normalBootCI1` the variance is used instead of the standard deviation.

The percentile bootstrap interval number 1 is correct, i.e. `percentBootCI <- quantile(bootEstB,probs=c(0.025,0.975))` and `percentBootCI2` is not correct.